LAPORAN PRAKTIKUM

STRUKTUR DATA

“Stack”

disusun Oleh:

Sofian Arba’i

2511533029

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur kehadirat Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum pekan 3 dengan judul "Stack" ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum serta sebagai sarana untuk meningkatkan pemahaman mengenai penggunaan struktur data Stack.

Dalam praktikum pekan 3 mempelajari dasar-dasar Stack, yaitu salah satu struktur data yang menggunakan prinsip LIFO (*Last In First Out*), di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Stack memiliki aturan khusus dalam proses penambahan dan penghapusan elemen. Selain itu, praktikum ini juga mempelajari berbagai operasi yang dapat dilakukan pada Stack, seperti *push*, *pop*, dan *peek*.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Strukdtur Data yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika banyak terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 13 April 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Stack 3
2.1.1 Kelebihan dan Kekurangan Stack 3
2.2 Impelementasi Stack Pada Java 4
2.2.1 Class NilaiMaksimum_2511533029 4
2.2.2 Class SiswaStack_2511533029 6
2.2.3 Class StackArray_2511533029 8
2.2.4 Class StackPostfix_2511533029 11
BAB III PENUTUP 13
3.1 Kesimpulan 13
3.2 Saran 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengolahan data yang efektif dan terorganisir. Dalam dunia pemrograman, pengelolaan data yang baik sangat diperlukan agar program dapat berjalan dengan efisien serta mudah untuk dikembangkan. Salah satu struktur data yang mendukung hal tersebut adalah Stack.

Stack merupakan struktur data yang bekerja dengan prinsip LIFO (*Last In First Out*), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Berbeda dengan struktur data lain yang lebih fleksibel, Stack memiliki aturan khusus dalam proses pengolahan datanya. Struktur ini sering digunakan dalam berbagai kasus seperti pengelolaan memori, undo-redo, serta evaluasi ekspresi dalam pemrograman.

Oleh karena itu, pemahaman mengenai Stack sangat penting bagi mahasiswa, khususnya dalam bidang struktur data dan pemrograman. Melalui praktikum ini, diharapkan mahasiswa dapat memahami konsep dasar Stack serta mampu mengimplementasikannya dalam berbagai permasalahan pemrograman

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 3 tentang Stack adalah sebagai berikut:

1. Mempelajari cara kerja struktur data Stack dalam pengelolaan data.
2. Mengembangkan kemampuan dalam merancang program menggunakan konsep Stack.

3. Menganalisis penggunaan Stack dalam menyelesaikan permasalahan tertentu pada pemrograman.
4. Mengetahui operasi-operasi dasar pada Stack seperti *push*, *pop*, dan *peek*.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 3 tentang Stack adalah sebagai berikut:

1. Memberikan pemahaman tentang penerapan Stack dalam kasus nyata di pemrograman.
2. Membantu meningkatkan keterampilan dalam mengelola data secara terstruktur.
3. Menjadi bekal dasar dalam mempelajari struktur data lainnya yang lebih kompleks.
4. Mengembangkan pola pikir logis dan sistematis dalam menyelesaikan masalah pemrograman.

BAB II
PEMBAHASAN

2.1 Pengertian Stack

Struktur data stack merupakan salah satu jenis struktur data pada komputer yang berguna untuk menyimpan dan mengorganisir data dalam urutan-urutan tertentu. Dengan menggunakan konsep LIFO, struktur data stack akan pertama kali mengambil data yang terakhir kali dimasukkan ke dalam stack. Jika dibayangkan, stack ini seperti sebuah tumpukan buku yang diletakkan dengan urutan atas ke bawah. Buku yang paling atas dan mudah diambil adalah buku yang akan diambil terlebih dahulu.

Dalam istilah pemrograman, usaha untuk menambahkan elemen pada sebuah struktur data stack disebut sebagai push. Adapun proses menghapus atau menghilangkan elemen data dari stack disebut sebagai pop. Stack sering dipakai dalam implementasi algoritma dan fungsi yang bersifat matematis serta konversi infix ke postfix. Selain itu, stack juga bisa dipakai dalam aplikasi yang membutuhkan manajemen tumpukan data seperti halnya pengembanga OS, kompilator, dan software grafis.

2.1.1 Kelebihan dan Kekurangan Stack

Berikut beberapa kelebihan dan kekurangan dari stack.

Kelebihan:

1. membantu mengelola data dengan metode LIFO
2. secara otomatis membersihkan objek
3. tidak mudah rusak
4. ukuran variabel tidak dapat diubah
5. mengontrol memori secara mandiri

Kekurangan:

1. memori *stack* sangat terbatas
2. ada kemungkinan *stack* akan meluap atau *overflow* jika terlalu banyak objek
3. tidak memungkinkan akses acak, karena harus mengeluarkan tumpukan paling atas terlebih dahulu untuk mengakses tumpukan paling bawah

2.2 Impelementasi Stack Pada Java

Pada praktikum pekan 3 mahasiswa diminta untuk mengikuti intruksi dari dosen, yaitu menuliskan kembali kode program yang sudah ada di layar. Kode program tentang nilai maksimum, siswa stack, stack array, stack array driver dan stackpostfix, berikut di bawah ini kode program yang dituliskan kembali ke dalam java.

2.2.1 Class NilaiMaksimum_2511533029

Kode program pertama yaitu kode program implementasi penggunaan stack dalam pemrograman java. Secara keseluruhan, program ini menunjukkan cara mencari nilai maksimum pada stack tanpa merusak isi data aslinya. Berikut kode program nilai maksimum:

```java
package pekan3_2511533029;
import java.util.Stack;
public class NilaiMaksimum_2511533029 {
    public static int max (Stack <Integer>s_3029) {
    Stack <Integer> backup_3029 = new Stack <Integer>();
    int maxValue_3029 = s_3029.pop();
    backup_3029.push (maxValue_3029);
    while (!s_3029.isEmpty()) {
        int next_3029=s_3029.pop();
        backup_3029.push(next_3029);
        maxValue_3029 = Math.max(maxValue_3029, next_3029);
    }
    while (!backup_3029.isEmpty() ) {
        s_3029.push(backup_3029.pop());
    }
    return maxValue_3029;
  }
    public static void main(String[] args) {
        Stack <Integer> s_3029 = new Stack<Integer> ();
        s_3029.push (70);
        s_3029.push (12);
        s_3029.push (20);
        System.out.println("isi stack " + s_3029);
        System.out.println("Stack Teratas " + s_3029.peek());
        System.out.println("nilai maksimum " + max(s_3029));
    }

}
```

Kode Program 2.1

Kode program di atas menggunakan struktur data Stack untuk mencari nilai maksimum dari sekumpulan data tanpa menghilangkan isi stack. Pada method main, dibuat sebuah objek stack bertipe Integer bernama s_3029, kemudian beberapa nilai dimasukkan menggunakan method push(), yaitu 70, 12, dan 20. Program lalu menampilkan isi stack, elemen teratas menggunakan peek(), serta nilai maksimum dengan memanggil method max().

Method max(Stack<Integer> s_3029) berfungsi untuk mencari nilai terbesar dalam stack. Pertama, dibuat stack sementara bernama backup_3029 untuk menyimpan elemen sementara agar data asli tidak hilang. Nilai pertama diambil dari stack menggunakan pop() dan dijadikan nilai awal maksimum, lalu disimpan ke stack backup. Selanjutnya, selama stack belum kosong, setiap elemen diambil satu per satu menggunakan pop(), disimpan ke backup, dan dibandingkan

dengan nilai maksimum menggunakan Math.max() untuk mendapatkan nilai terbesar. Setelah semua elemen diproses, isi stack dikembalikan ke kondisi semula dengan memindahkan kembali elemen dari stack backup ke stack utama. Terakhir, method ini mengembalikan nilai maksimum yang ditemukan.

2.2.2 Class SiswaStack_2511533029

Kode program kedua yaitu kode program untuk menunjukkan cara kerja stack dengan prinsip LIFO (*Last In First Out*) menggunakan objek sebagai data yang disimpan. Berikut kode program Siswa Stack:

```java
package pekan3_2511533029;
import java.util.ArrayList;
class Siswa_2511533029 {
    String nama_3029;
    int nim_3029;

    public Siswa_2511533029 (String nama_3029, int nim_3029) {
        this.nama_3029= nama_3029;
        this.nim_3029= nim_3029;

    }

    @Override
    public String toString () {
        return "NIM: " + nim_3029 + ", Nama : " +nama_3029;
    }
}
public class SiswaStack_2511533029 {
    private ArrayList<Siswa_2511533029> stack_3029;

    public SiswaStack_2511533029() {
        stack_3029 = new ArrayList<>();
    }
    public void push (Siswa_2511533029 mhs) {
        stack_3029.add(mhs);
    }

public Siswa_2511533029 pop() {
    if (!isEmpty()) {
        return stack_3029.remove(stack_3029.size() -1);
     }
     return null;
    }
 public Siswa_2511533029 peek() {
     if (!isEmpty()) {
         return stack_3029.get(stack_3029.size() -1 );
     }
     return null;
 }

```

Kode Program 2.2

```java
public boolean isEmpty () {
    return stack_3029.isEmpty();
    }
 public void tampilkanSiswa_3029() {
     for (int i= stack_3029.size() -1; i>=0; i--) {
         System.out.println(stack_3029.get(i));
     }
 }
 public static void main (String [] args) {
     SiswaStack_2511533029 studentstack = new SiswaStack_2511533029();

     Siswa_2511533029 mhs1 = new Siswa_2511533029 ("Ali ", 1);
     Siswa_2511533029 mhs2 = new Siswa_2511533029 ("Boby ", 2);
     Siswa_2511533029 mhs3 = new Siswa_2511533029 ("Charles ", 3);

     studentstack.push(mhs1);
     studentstack.push(mhs2);
     studentstack.push(mhs3);

     System.out.println("siswa di dalam stack : ");
     studentstack.tampilkanSiswa_3029();

     System.out.println("siswa teratas " + studentstack.peek());
     System.out.println("mengeluarkan siswa teratas dari stack " + studentstack.pop());
     System.out.println("daftar siswa setelah di pop : ");
     studentstack.tampilkanSiswa_3029();
 }

}
```

Kode Program 2.3

Kode program 2.3 merupakan implementasi struktur data Stack menggunakan ArrayList untuk menyimpan objek bertipe Siswa_2511533029. Class Siswa_2511533029 digunakan untuk merepresentasikan data siswa yang memiliki atribut nama_3029 dan nim_3029, yang diinisialisasi melalui konstruktor. Method toString() dioverride agar data siswa dapat ditampilkan dalam format yang rapi saat dicetak.

Class SiswaStack_2511533029 berfungsi sebagai implementasi stack, dengan atribut stack_3029 bertipe ArrayList sebagai tempat penyimpanan data. Konstruktor digunakan untuk menginisialisasi ArrayList kosong. Method push() digunakan untuk menambahkan data siswa ke bagian atas stack, sedangkan pop() berfungsi menghapus dan mengembalikan elemen teratas (elemen terakhir dalam ArrayList). Method peek() digunakan untuk melihat elemen teratas tanpa menghapusnya, dan isEmpty() digunakan untuk mengecek apakah stack kosong. Selain itu, method tampilkanSiswa_3029() digunakan untuk menampilkan seluruh isi stack dari atas ke bawah dengan melakukan iterasi dari indeks terakhir ke awal.

Pada method main, dibuat beberapa objek siswa, kemudian dimasukkan ke dalam stack menggunakan push(). Program kemudian menampilkan seluruh isi stack, menampilkan elemen teratas dengan peek(), menghapus elemen teratas dengan pop(), dan akhirnya menampilkan kembali isi stack setelah terjadi perubahan.

2.2.3 Class StackArray_2511533029

Kode program kedua yaitu kode program yang menunjukkan implementasi dasar stack menggunakan array dengan prinsip LIFO (*Last In First Out*). Berikut kode program stack array:

```java
package pekan3_2511533029;

public class stackArray_2511533029 {
    static final int MAX_3029= 1000;
    int top_3029;
    int a[] = new int [MAX_3029];
    boolean isEmpty_3029()
    {
        return (top_3029 < 0) ;
    }
    stackArray_2511533029()
    {
        top_3029=-1;
    }
    boolean push_3029 (int x)
    {
        if (top_3029>= (MAX_3029 - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a[++top_3029] = x;
            System.out.println( x  + " dimasukkan dalam stack");
            return true;
        }
    }
```

KodeProgram 2.4

```
    int pop_3029()
    {
        if (top_3029 <= 0 ) {
            System.out.println("stack underflow");
            return 0;
        }else {
            int x = a[top_3029--];
            return x;
        }
    }
    int peek_3029()
    {
        if (top_3029<0)
        {
            System.out.println("stack underflow");
            return 0;
        }
        else
        {
            int x = a [top_3029];
            return x;
        }
    }
    void print_3029 () {
        for (int i=top_3029; i>-1;i--) {
            System.out.print(" " +a[i]);
        }
    }

}
```

Kode Program 2.5

Kode program merupakan implementasi struktur data Stack menggunakan array sebagai media penyimpanan. Pada class stackArray_2511533029, didefinisikan sebuah array a dengan kapasitas maksimum 1000 elemen dan variabel top_3029 yang berfungsi sebagai penunjuk posisi elemen teratas dalam stack. Konstruktor menginisialisasi nilai top_3029 menjadi -1 yang menandakan bahwa stack masih kosong. Method isEmpty_3029() digunakan untuk mengecek apakah stack kosong, yaitu ketika nilai top_3029 kurang dari 0.

Operasi utama stack diimplementasikan melalui beberapa method. Method push_3029(int x) digunakan untuk menambahkan elemen ke dalam stack dengan cara menaikkan nilai top_3029 lalu menyimpan data ke array, serta dilengkapi pengecekan kondisi

overflow jika stack sudah penuh. Method pop_3029() digunakan untuk menghapus dan mengembalikan elemen teratas dengan menurunkan nilai top_3029, serta menangani kondisi *underflow* jika stack kosong. Method peek_3029() berfungsi untuk melihat elemen teratas tanpa menghapusnya, sedangkan print_3029() digunakan untuk menampilkan seluruh isi stack dari atas ke bawah.

Untuk menjalankan kode program 2.4 dan 2.5 dibutuhkan suatu kode program lagi dengan class berbeda. Kode program ini disebut sebagai *driver* atau penguji, yaitu menjalankan program untuk mencoba dan menampilkan hasil dari setiap operasi yang telah disediakan pada class StackArrayDriver_2511533029. Berikut kode program dari class driver StackArrayDriver_2511533029:

```
1  package pekan3_2511533029;
2
3  public class stackArrayDriver_2511533029 {
4
5      public static void main(String[] args) {
6          stackArray_2511533029 s = new stackArray_2511533029 ();
7          s.push_3029(10);
8          s.push_3029(20);
9          s.push_3029(30);
10         System.out.println(s.pop_3029() + " dikeluarkan dari stack");
11         System.out.println("elemen teratas adalah : " + s.peek_3029());
12         System.out.print("element pada stack    :");
13         s.print_3029();
14
15     }
16
17 }
```

Kode Program 2.6

Class stackArrayDriver_2511533029 berfungsi sebagai penguji dari class stackArray. Pada method main, dibuat objek stack kemudian dilakukan beberapa operasi seperti menambahkan elemen menggunakan push_3029(), menghapus elemen teratas dengan pop_3029(), melihat elemen teratas dengan peek_3029(), dan menampilkan seluruh isi stack menggunakan print_3029(). Class ini menunjukkan bagaimana cara penggunaan method-method pada stack serta menggambarkan cara kerja stack dengan prinsip LIFO (*Last In First Out*)

2.2.4 Class StackPostfix_2511533029

Kode program terakhir pada pekan 3 yaitu kode program yang menunjukkan penerapan stack dalam menyelesaikan perhitungan ekspresi postfix dengan prinsip LIFO (*Last In First Out*). Berikut kode program stack post fix:

```java
package pekan3_2511533029;
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511533029 {

    public static int postfixEvaluate_3029 (String expression) {
        Stack <Integer> s = new Stack <Integer>();
        Scanner input_3029 = new Scanner (expression);
        while (input_3029.hasNext()) {
            if (input_3029.hasNextInt()) {  //an operan integer
                s.push(input_3029.nextInt());
            } else {
                String operator_3029 = input_3029.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator_3029.equals("+")) {
                    s.push (operand1+operand2);
                } else if (operator_3029.equals("-")) {
                    s.push(operand1-operand2);
                } else if (operator_3029.equals("*")) {
                    s.push(operand1*operand2);
                } else {
                    s.push(operand1/operand2);
                }
            }
        }
        input_3029.close();
        return s.pop();
    }
    public static void main (String [] args) {
        System.out.println("hasil postfix = " + postfixEvaluate_3029("5 2 5 * + 7 -"));
    }

}
```

Kode Program 2.7

Kode program di atas digunakan untuk menghitung hasil dari ekspresi postfix (Reverse Polish Notation) dengan memanfaatkan struktur data Stack. Method postfixEvaluate_3029(String expression) berfungsi sebagai inti program yang menerima ekspresi dalam bentuk string, kemudian memprosesnya menggunakan objek Scanner untuk membaca setiap token (angka atau operator). Jika token berupa bilangan bulat (hasNextInt()), maka nilai tersebut dimasukkan ke dalam stack menggunakan push(). Namun jika token berupa operator, program akan mengambil dua nilai teratas dari stack menggunakan pop(), kemudian melakukan operasi sesuai operator tersebut, seperti

penjumlahan (+), pengurangan (-), perkalian (*), atau pembagian (/). Hasil dari operasi tersebut kemudian dimasukkan kembali ke dalam stack.

Proses ini dilakukan berulang hingga seluruh ekspresi selesai dibaca. Setelah itu, nilai terakhir yang tersisa di dalam stack merupakan hasil akhir dari perhitungan postfix dan dikembalikan oleh method. Pada method main, program memanggil fungsi tersebut dengan contoh ekspresi "5 2 5 * + 7 -", kemudian menampilkan hasilnya.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan pembahasan dan praktikum yang telah dilakukan, dapat disimpulkan bahwa Stack merupakan struktur data yang bekerja dengan prinsip LIFO (*Last In First Out*), di mana elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Stack memiliki operasi dasar yaitu *push* untuk menambahkan elemen, *pop* untuk menghapus dan mengambil elemen teratas, serta *peek* untuk melihat elemen teratas tanpa menghapusnya. Melalui praktikum ini, telah dipelajari berbagai implementasi Stack dalam bahasa Java, termasuk pencarian nilai maksimum dalam stack, pengelolaan data objek mahasiswa, implementasi stack menggunakan array, serta penerapan stack dalam evaluasi ekspresi postfix. Stack memiliki kelebihan seperti pengelolaan data yang terstruktur, manajemen memori yang mandiri, dan tidak mudah rusak, namun juga memiliki keterbatasan seperti ukuran memori yang terbatas, risiko *overflow*, dan tidak memungkinkan akses acak ke elemen. Pemahaman terhadap konsep Stack sangat penting sebagai dasar untuk mempelajari struktur data yang lebih kompleks.

3.2 Saran

Untuk meningkatkan pemahaman dan penguasaan materi Stack, disarankan agar mahasiswa terus berlatih mengimplementasikan Stack dalam berbagai kasus pemrograman yang lebih kompleks dan bervariasi. Selain itu, penting untuk memperdalam pemahaman tentang perbedaan penggunaan Stack dengan struktur data lainnya seperti ArrayList, serta mengetahui kapan waktu yang tepat untuk menggunakan masing-masing struktur data tersebut. Disarankan juga untuk lebih teliti dalam menangani kondisi *overflow* dan *underflow* saat mengimplementasikan stack.

DAFTAR PUSTAKA

[1] Najmaa Shiba, “Struktur Data Stack: Pengertian, Jenis dan Contoh,”. [Daring]. Tersedia pada: **Struktur Data Stack: Pengertian, Jenis dan Contoh - IDS Digital College**[Diakses: 14-Apr-2026].

[2] Maulana Adieb, “Memahami Stack, Struktur Data yang Mudahkan Programmer Jalankan Tugasnya,”. [Daring]. Tersedia pada: **Stack: Pengertian, Kelebihan dan Kekurangan serta Jenis Operasinya**[Diakses: 14-Apr-2026].

LAPORAN TUGAS PRAKTIKUM

Struktur Data

Pekan 3

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Topik : Implementasi Stack (LIFO) menggunakan java.util.Stack

Tema : Simulasi Browser History

1. Deskripsi Tugas

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

2. Aturan Penamaan (Wajib Dipatuhi)

1) Nama Kelas : Gunakan akhiran NIM lengkap.
   Contoh : Website_2311532008 dan Browser_2311532008.

2) Nama Variabel Atribut : Setiap variabel atribut di kelas ADT gunakan akhiran 4 digit terakhir NIM.
   Contoh (NIM ...2008) : String judul_2008, String url_2008..

3. Struktur Program

1) Kelas ADT Website (Website_NIMLengkap)
   Atribut : Judul Website, URL Website.

   Method :
   - Constructor untuk menginisialisasi atribut.
   - Selektor (Getter) untuk setiap atribut.
   - Mutator (Setter) untuk setiap atribut.

2) Kelas Driver (Browser_NIMLengkap)
   - Kunjungi Website (Push) : Memasukkan data website baru yang sedang dikunjungi ke dalam tumpukan.

- Tombol Back (Pop) : Menghapus halaman website terakhir (paling atas) dan menampilkan judul halaman yang berhasil dihapus.
- Lihat Halaman Saat Ini (Peek) : Menampilkan detail website yang sedang aktif dibuka tanpa menghapusnya dari tumpukan.
- Cek Status History : Menampilkan jumlah total riwayat yang tersimpan atau mengecek apakah riwayat kosong menggunakan .isEmpty()

4. Contoh Tampilan Output Program

```
=== Browser History NIM: 2311532008 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1

Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login

Berhasil mengunjungi halaman!
```

JAWABAN

1. Kode Program

1.1 Class Website_2511533029

```java
package pekan3_2511533029;

public class Website_2511533029 {
    String judulWebsite_3029;
    String URLWebsite_3029;

    //Konstruktor
    public Website_2511533029(String judulWebsite_3029, String URLWebsite_3029) {
        this.judulWebsite_3029 = judulWebsite_3029;
        this.URLWebsite_3029 = URLWebsite_3029;
    }

    //Selektor
    public String getjudulWebsite_3029() { return judulWebsite_3029; }
    public String getURLWebsite_3029() { return URLWebsite_3029;}


    //Mutator
    public void setjudulWebsite_3029 (String judulWebsite_3029) {
        this.judulWebsite_3029 = judulWebsite_3029;
    }
    public void setURLWebsite_3029 (String URLWebsite_3029) {
        this.URLWebsite_3029 = URLWebsite_3029;
    }

    @Override
    public String toString () {
        return "Judul Website : " + judulWebsite_3029 + ", URL Website : " + URLWebsite_3029;

    }

}
```

Kode Program 1.1

Class Website_2511533029 digunakan sebagai ADT untuk menyimpan data website yang terdiri dari dua atribut, yaitu judulWebsite_3029 dan URLWebsite_3029 yang bertipe String. Atribut judulWebsite_3029 digunakan untuk menyimpan nama atau judul website, sedangkan URLWebsite_3029 digunakan untuk menyimpan alamat website. Class ini memiliki konstruktor Website_2511533029(String judulWebsite_3029, String URLWebsite_3029) yang berfungsi untuk menginisialisasi nilai atribut saat objek dibuat. Selain itu, terdapat method selektor atau getter yaitu getjudulWebsite_3029() dan getURLWebsite_3029() yang digunakan untuk mengambil nilai atribut. Terdapat juga method mutator atau setter yaitu setjudulWebsite_3029() dan setURLWebsite_3029() yang digunakan untuk

mengubah nilai atribut. Pada bagian akhir, class ini memiliki method toString() yang berfungsi untuk menampilkan data website dalam bentuk teks sehingga memudahkan saat objek dipanggil atau ditampilkan ke layar.

1.2 Class Browser_2511533029

```java
package pekan3_2511533029;
import java.util.Stack;
import java.util.Scanner;

public class Browser_2511533029 {
    //1. Tampilkan Menu
    public static void tampilkanMenu_3029 () {
    System.out.println("\n=== Browser History NIM: 2511533029 === ");
    System.out.println("1. Kunjungi Website (Push) ");
    System.out.println("2. Tombol Back (Pop) ");
    System.out.println("3. Lihat Halaman Aktif (Peek) ");
    System.out.println("4. Keluar");
}
//2. Method Untuk Kunjungi Website (Push)
    public static void kunjungiWebsite_3029(Stack<Website_2511533029> stack, Scanner sc) {
    System.out.print("\nMasukkan Judul Website: ");
    String judulWebsite_3029= sc.nextLine();

    System.out.print("Masukkan URL: ");
    String URLWebsite_3029= sc.nextLine();

    stack.push(new Website_2511533029(judulWebsite_3029, URLWebsite_3029));
    System.out.println(" Berhasil Mengunjungi Halaman");
}
//3. Method Untuk Tombol Back (Pop)
    public static void tombolBack_3029(Stack<Website_2511533029> stack) {
        if (stack.isEmpty()) {
            System.out.println("History Kosong.");
        } else {
            Website_2511533029 website = stack.pop();
            System.out.println("Kembali dari website: " + website.judulWebsite_3029);
            }
        }

//4. Method Untuk Lihat Halaman Aktif (Peek)
    public static void lihathalamanAktif_3029(Stack<Website_2511533029> stack) {
        if (stack.isEmpty()) {
            System.out.println("Halaman Aktif Kosong");
        } else {
            Website_2511533029 website = stack.peek();
            System.out.println("Halaman Yang Sedang Aktif: " + website);
```

Kode Program 1.2

```java
            System.out.println("Halaman Yang Sedang Aktif: " + website);
        }
    }
//5. Method Utama Program
    public static void main (String [] args) {
        Stack <Website_2511533029> websiteStack = new Stack <>();
        Scanner scanner = new Scanner (System.in);
        int choice;
        do {
            tampilkanMenu_3029();
            System.out.print("Pilihan: ");
            choice = scanner.nextInt();
            scanner.nextLine(); //consume newline

            switch (choice ) {
            case 1:
                kunjungiWebsite_3029(websiteStack, scanner);
                break;
            case 2:
                tombolBack_3029(websiteStack);
                break;
            case 3:
                lihathalamanAktif_3029(websiteStack);
                break;
            case 4:
                System.out.println(" Keluar dari program.");
                break;
                default:
                System.out.println("pilihan tidak valid. ");
            }
        }
        while (choice !=4);
        scanner.close();
    }
}
```

Kode Program 1.3

Class Browser_2511533029 digunakan untuk membuat program sederhana browser history menggunakan struktur data Stack. Program ini mengimpor Stack untuk menyimpan riwayat website dan Scanner untuk menerima input dari pengguna. Pada method tampilkanMenu_3029(), program menampilkan menu utama yang berisi pilihan untuk mengunjungi website, menggunakan tombol back, melihat halaman aktif, dan keluar dari program.

Method kunjungiWebsite_3029() digunakan untuk menambahkan website baru ke dalam stack dengan operasi push(). Pengguna diminta memasukkan judul website dan URL, kemudian data tersebut disimpan dalam objek Website_2511533029 dan dimasukkan ke stack. Method tombolBack_3029() digunakan untuk menghapus website paling atas menggunakan operasi pop(). Jika stack kosong, program menampilkan pesan

bahwa history kosong. Sedangkan method lihathalamanAktif_3029() digunakan untuk melihat website yang sedang aktif menggunakan operasi peek() tanpa menghapus data dari stack.

Pada method main(), program membuat stack bernama websiteStack, scanner untuk input, dan variabel choice untuk menyimpan pilihan menu. Program berjalan menggunakan perulangan do-while sehingga menu akan terus tampil sampai pengguna memilih menu keluar. Setiap pilihan menu diproses menggunakan switch-case, sehingga pengguna dapat menambahkan website, menghapus website terakhir, melihat halaman aktif, atau menghentikan program.

2. Output Kode Program

Berikut output dari kode program yang saya buat dengan menjelaskan masing-masing menu:

```
=== Browser History NIM: 2511533029 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1

Masukkan Judul Website: Ilearn
Masukkan URL: https://it.ilearn.unand.ac.id/
 Berhasil Mengunjungi Halaman

=== Browser History NIM: 2511533029 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
Kembali dari website: Ilearn

=== Browser History NIM: 2511533029 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Halaman Aktif Kosong

=== Browser History NIM: 2511533029 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 4
 Keluar dari program.
```

Output 2.1

Berikut juga output jika tidak mengunjungi satu website manapun:

```
=== Browser History NIM: 2511533029 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
History Kosong.

=== Browser History NIM: 2511533029 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Halaman Aktif Kosong

=== Browser History NIM: 2511533029 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 4
 Keluar dari program.
```

Output 2.2